Meddy: An AI Medical Chatbot for Symptom-Based Disease Prediction

Research Paper

Abstract

This paper presents Meddy, an AI medical chatbot designed to collect user-reported symptoms, interpret natural-language input, ask relevant follow-up questions, and predict a possible disease using machine learning. The motivation behind the project is the need for an accessible educational health assistant that can guide users toward understanding possible conditions while clearly remaining non-diagnostic and not replacing professional medical care. The problem addressed is that users often describe symptoms in informal, misspelled, or incomplete language, making direct disease prediction difficult. To address this issue, Meddy combines natural language processing, spell correction, a PyTorch neural network for symptom recognition, and a scikit-learn stacking classifier for disease prediction. The system also uses disease descriptions, precaution data, symptom severity scores, confidence thresholds, and emergency warnings to make the interaction more informative and safer. Experiments were conducted through model training on a disease-symptom dataset, train-test splitting, internal five-fold cross-validation inside the stacking classifier, and functional tests for spelling correction, greeting handling, unknown input, and symptom extraction. The experimental findings show that the system can transform user phrases into symptom tags, combine multiple symptoms into a one-hot encoded vector, generate follow-up questions, and produce disease predictions with descriptions and precautions. The work demonstrates how a combined NLP and machine learning pipeline can support interactive symptom assessment while also highlighting limitations related to dataset quality, real-world clinical reliability, and evaluation methodology.

1. Introduction

Healthcare information is frequently searched online before users decide whether they need professional help. However, general search engines often return broad or confusing results, and users may not know how to translate everyday descriptions such as “my chest hurts” or “I feel like throwing up” into structured medical symptoms. This creates a practical need for conversational tools that can collect symptom information in a simpler and more organized way. The Meddy chatbot was developed to address this need by acting as an educational medical assistant that receives symptom descriptions, identifies possible symptoms, asks follow-up questions, and suggests a possible disease based on a trained model.

The project focuses on symptom-based disease prediction rather than full medical diagnosis. This distinction is important because a real diagnosis requires physical examination, patient history, laboratory tests, imaging, and professional clinical judgment. Meddy instead works as a software prototype that demonstrates how artificial intelligence and machine learning can be combined with a web interface to support a structured health conversation. The system is designed to be user-friendly: the user does not need to know the exact dataset symptom names and can instead write natural sentences such as “I have a headache and I feel dizzy.”

The main contribution of this project is the integration of two AI components. First, a PyTorch neural network processes natural-language symptom descriptions and maps them to symptom tags. Second, a scikit-learn disease prediction model receives the collected symptom tags as one-hot encoded input and predicts the most likely disease. Around these two models, the Flask application manages the conversation flow, spell correction, follow-up questions, severity checking, confidence handling, descriptions, precautions, emergency warnings, and chat history. The final system therefore behaves as a conversation-based prediction tool rather than a single-step classifier.

2. Related Work

Medical chatbots generally combine conversational interfaces with knowledge-based rules, natural language processing, or machine learning models. In rule-based systems, the chatbot follows fixed questions and predefined decision paths. These systems are easy to control but may fail when users write symptoms in unexpected language. Machine learning systems can be more flexible because they learn patterns from examples, but they also depend heavily on the quality and representativeness of the training data. Meddy combines both ideas: machine learning is used for symptom and disease prediction, while rule-like logic is used for confidence thresholds, emergency warnings, and follow-up question control.

Natural language processing is important in chatbot systems because user input is rarely structured. A user may write “I puked,” “I threw up,” or “I feel like vomiting,” and all of these should map to the same symptom. In this project, symptom phrases are stored in intent files where each symptom tag is connected to several example patterns. The text is tokenized, stemmed, converted into a bag-of-words vector, and passed into a neural network. This approach is simpler than transformer-based language models, but it is appropriate for a controlled academic project because the symptom categories are known in advance.

Disease prediction from symptoms is commonly treated as a supervised classification problem. Each disease becomes a class label, and each symptom becomes a feature. In Meddy, the dataset is transformed into a one-hot encoded table where each symptom column contains either 1 or 0. Ensemble learning is then used through a stacking classifier. Stacking is useful because it combines the output of several different models and trains a final estimator to make the final prediction. In this implementation, Logistic Regression, K-Nearest Neighbors, Decision Tree, and Support Vector Machine models are combined, with Logistic Regression used as the final estimator.

3. Methods

3.1 System Architecture

Meddy is implemented as a Flask web application. The application loads the trained models and data files at startup, then waits for user messages through a symptom route. The main system components are the web interface, the natural language symptom recognizer, the disease prediction model, and the response generation module. The web interface collects user input and displays chatbot responses. The symptom recognizer identifies symptom tags from user text. The disease prediction model converts confirmed symptoms into a vector and predicts a disease. The response generation module adds descriptions, precautions, confidence messages, severity advice, and emergency warnings.

The application uses several data files. The main disease-symptom dataset and supporting symptom severity, disease description, and precaution files were obtained from the public Disease Symptom Prediction dataset on Kaggle (itachi9604, n.d.). The disease-symptom dataset stores diseases with their related symptoms. The symptom description file stores explanations of each disease. The precaution file stores recommended precautions for each disease. The symptom severity file assigns weights to symptoms, allowing the chatbot to detect severe cases. A symptom list stored as a pickle file is used to keep a consistent feature order when building model input vectors. Chat history is saved in JSON format after each final prediction.

Component	Purpose
Flask application	Hosts the chatbot, receives messages, and returns responses.
PyTorch neural network	Classifies user phrases into symptom tags.
Bag-of-words NLP module	Converts tokenized and stemmed words into numeric vectors.
Stacking classifier	Predicts disease from one-hot encoded symptoms.
CSV medical files	Provide disease symptoms, descriptions, precautions, and severity weights.
Spell checker	Corrects misspelled symptom words before prediction.
Chat history JSON	Stores timestamp, selected symptoms, and predicted disease.

3.2 Natural Language Processing and Symptom Recognition

The natural language processing stage begins by converting the user sentence to lowercase and tokenizing it into words. A spell checker is then used to correct mistakes such as “vomting” or “cheest.” Medical symptom words are added to the spell checker dictionary with high frequency so that valid symptom terms are protected from incorrect correction. The corrected sentence is split into smaller phrases using punctuation and conjunctions such as “and” and “but,” allowing multiple symptoms to be extracted from a single message.

The system identifies symptoms using two methods. First, it searches for exact symptom matches by replacing underscores in stored symptom names and checking whether the cleaned symptom phrase appears in the user input. If exact matching fails, the phrase is passed into the neural network symptom classifier. The phrase is tokenized, converted into a bag-of-words vector, and classified into one of the symptom tags. A probability threshold is used so that only symptom predictions above the threshold are accepted. This hybrid approach improves robustness

because exact symptom phrases are detected directly, while informal descriptions can still be recognized by the neural network.

3.3 Disease Prediction Model

The disease prediction model is trained using a supervised classification approach. The original dataset contains a disease label and multiple symptom columns. During preprocessing, all unique symptoms are collected from the symptom columns. Each unique symptom is then converted into its own feature column. For each disease row, the feature value is set to 1 if the symptom is present and 0 otherwise. This produces a one-hot encoded representation that machine learning algorithms can process.

The final classifier is a stacking ensemble. The base models include Logistic Regression, K-Nearest Neighbors with cosine distance, a Decision Tree classifier, and a Support Vector Classifier. A Logistic Regression model acts as the final estimator. The stacking classifier uses five-fold cross-validation internally, meaning the final estimator is trained using cross-validated predictions from the base estimators. This design attempts to benefit from the strengths of different algorithms: Logistic Regression handles linear decision patterns, KNN compares similar symptom vectors, Decision Trees capture rule-like structures, and SVM helps separate classes in feature space.

4. Experimental Design

The experimental design evaluates both the model training process and the chatbot behavior. For model training, the disease-symptom dataset is loaded using Pandas, duplicate rows are removed, and symptoms are transformed into one-hot encoded features. The data is split using an 80/20 train-test split with a fixed random state. The stacking model is configured with five-fold cross-validation inside the ensemble. After training, the model is scored on the test portion and saved as a pickle file for later use by the Flask application.

A second experimental layer focuses on functional behavior. Several tests were created to check important chatbot cases. A spelling test checks whether a misspelled symptom can be corrected. A logic test sends a mixed sentence containing a greeting and misspelled symptoms, then checks whether the input can be corrected, split, and mapped to symptoms. A zero-input test checks how the symptom recognition function behaves when the phrase is a greeting or an unknown word. These tests are important because chatbot quality depends not only on final accuracy but also on input handling, conversation control, and error recovery.

At runtime, the system also performs a practical confidence experiment. When the user types “Done,” the model calculates class probabilities when available. If the maximum probability is below 0.8, the chatbot does not immediately present the diagnosis and instead asks the user for more symptoms. If the user has no additional symptoms, the user can type “Force Done” to receive the prediction anyway. This confidence mechanism is not a substitute for clinical validation, but it is useful as a safety-oriented design feature because it reduces the chance that the chatbot gives a confident-sounding answer when the model is uncertain.

5. Experimental Results

The implemented system successfully loads the required models and medical data, receives user messages, extracts symptoms, asks follow-up questions, and produces a disease prediction. The chat history file shows recorded examples in which headache was predicted as migraine, continuous sneezing with chills was predicted as allergy, and chest pain with vomiting and acidity was predicted as GERD. These examples demonstrate that the end-to-end pipeline works from symptom collection to final history storage.

The symptom extraction process also supports realistic user input. For example, the logic test uses the sentence “Hello Meddy! I am feeling terrible. I have cheest pain and vomting.” This sentence contains a greeting, emotional language, and misspelled symptoms. The system corrects misspelled words, splits the sentence into phrases, ignores greetings, and attempts to classify the remaining phrases into symptom tags. This shows that the chatbot is designed for imperfect natural input rather than only clean dataset terms.

The model training process produces a saved disease prediction model. The model is trained on one-hot encoded symptom features and uses a stacking classifier with five-fold cross-validation internally. However, one limitation appears in the training code: although a train-test split is created, the final fit call uses the full feature matrix rather than only the training subset. Therefore, the printed test score may be optimistic because the model has been exposed to the examples before scoring. A more rigorous experiment would fit the model only on the training set and evaluate on the held-out test set, then report accuracy, precision, recall, F1-score, and a confusion matrix.

From a system-design perspective, the main successful result is the chatbot's conversational behavior. It stores accepted symptoms, stores rejected symptoms, avoids asking rejected symptoms again, asks at most three follow-up questions, and uses predicted disease probabilities to choose whether to continue asking questions or request the user to type "Done." When the final disease is predicted, the chatbot retrieves a matching disease description and four precautions. If the predicted condition belongs to the critical disease list, it displays an emergency warning. Otherwise, it checks symptom severity weights and recommends consulting a doctor when symptoms appear serious.

6. Conclusions

This project demonstrates that a medical chatbot can be built by combining a web application, NLP preprocessing, neural network symptom recognition, and supervised disease classification. Meddy receives natural-language symptom descriptions, corrects spelling mistakes, maps phrases to symptom tags, asks follow-up questions, and predicts a possible disease using a stacking classifier trained on one-hot encoded symptom data. The main takeaway is that the strongest version of the system is not only the prediction model but the complete conversation pipeline around it: confidence checking, yes/no handling, severity warnings, emergency alerts, disease descriptions, precautions, and history storage. At the same time, the project remains a prototype and should be presented as an educational support tool, not a replacement for medical professionals. Future work should improve the evaluation method, use larger clinically validated datasets, add patient context such as age and medical history, and report stronger metrics such as precision, recall, F1-score, and confusion matrices.

7. Bibliography

itachi9604. (n.d.). Disease Symptom Prediction: dataset.csv. Kaggle. Retrieved May 18, 2026, from https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset/data?select=dataset.csv

itachi9604. (n.d.). Disease Symptom Prediction: Symptom-severity.csv. Kaggle. Retrieved May 18, 2026, from https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset/data?select=Symptom-severity.csv

itachi9604. (n.d.). Disease Symptom Prediction: symptom_Description.csv. Kaggle. Retrieved May 18, 2026, from https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset/data?select=symptom_Description.csv

itachi9604. (n.d.). Disease Symptom Prediction: symptom_precaution.csv. Kaggle. Retrieved May 18, 2026, from https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset/data?select=symptom_precaution.csv

Flask Documentation. “Welcome to Flask.” Pallets Projects, 2026. https://flask.palletsprojects.com/

PyTorch Documentation. “PyTorch Documentation.” PyTorch Foundation, 2026. https://docs.pytorch.org/docs/stable/index.html

Scikit-learn Documentation. “sklearn.ensemble.StackingClassifier.” Scikit-learn Developers, 2026. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html

NLTK Project. “Natural Language Toolkit Documentation.” NLTK Project. https://www.nltk.org/

Meddy Project Source Code. app.py, retrain_model.py, nnet.py, nltk_utils.py, intents.json, dataset.csv, symptom_Description.csv, symptom_precaution.csv, Symptom-severity.csv, and chat_history.json.